keeping vertices from drifting apart, but for a dense graph, if β>δ then all the vertices will end up converging to the center of the graph.

A possible alternative is deciding, based on the vertices/edges in the graph and on the size of the canvas where we embed the graph, what's the ideal length for an edge (or an ideal range for such length): this way, optimization will move away from solutions where all the vertices are clustered too closely.

$$\bar{J}(V) = \delta \cdot \sum_{v \epsilon V} \sum_{u \epsilon V/\{v\}} \|u - v\|_2^2 - \beta \cdot \sum_{v \epsilon V} \sum_{u \epsilon \text{adj}(v)} \left[\|u - v\|_2 - (\text{ideal edge length})\right]^2$$

Neither of these cost functions aims to directly reduce the number of intersections, but – as you can imagine - having shorter edges and keeping adjacent vertices close to each other indirectly help driving it down. And neither function is ideal, because they are not bowl-shaped and as such they will have several local minima. While we can't easily correct this shortcoming, we still have a workaround: we can use a random-restart algorithm, randomly selecting an initial position for the vertices, and the surfing the cost function downhill with gradient descent.

16.5.2 Implementation

Perseverance is the key, so if we repeat gradient descent step a few times (or maybe a lot of times? It really depends on the context!), starting from different positions and perhaps even with different learning rates, the final result might not be that bad.

In next chapter, we'll see a more sophisticated technique to make our optimization more flexible and raise our chances to land a good result.